

Networks, Maps, and Connections - Modeling the Real World

Lecture 19

Data Structures and Algorithms

Welcome to the Second Part of the Unit

We have concluded our journey with **Hash Tables**. You now know how to map data and utilize $O(1)$ structures across compiling algorithms, cache management, and memory systems.

Today, we transition to the second major part of the course's unified topics: **Graphs**.

Question: What do a city's road network, your Facebook friends list, the physical layout of the World Wide Web, and a computer circuit board all have in common?

Answer: They are all non-hierarchical interconnected models.
Today, we learn how to map the *real world* into data.

Part 1: What is a Graph?

Motivation: Beyond Trees

Let's look back at **Trees**.

Trees perfectly model hierarchical relationships. A CEO, followed by Managers, followed by Employees. A folder containing sub-folders containing files. A node has exactly one parent, creating exactly one path between the root and any node.

But what if we try to map Facebook?

- Alice is friends with Bob.
- Bob is friends with Carol.
- Carol is also friends with Alice.

A tree cannot model this! It creates a looped relationship (a cycle), and there is no strict "parent vs child" hierarchy. A social relationship is mutual.

To model this, we use a **Graph**.

Definition of a Graph

A **Graph** is the most general structure for modeling pairwise relationships between objects.

Formally, a Graph $G = (V, E)$, where:

- V is a set of **Vertices** (or Nodes). These represent the entities (e.g., people, cities, computers).
- E is a set of **Edges** (or connections). These represent the relationships between the entities (e.g., friendships, roads, network cables).

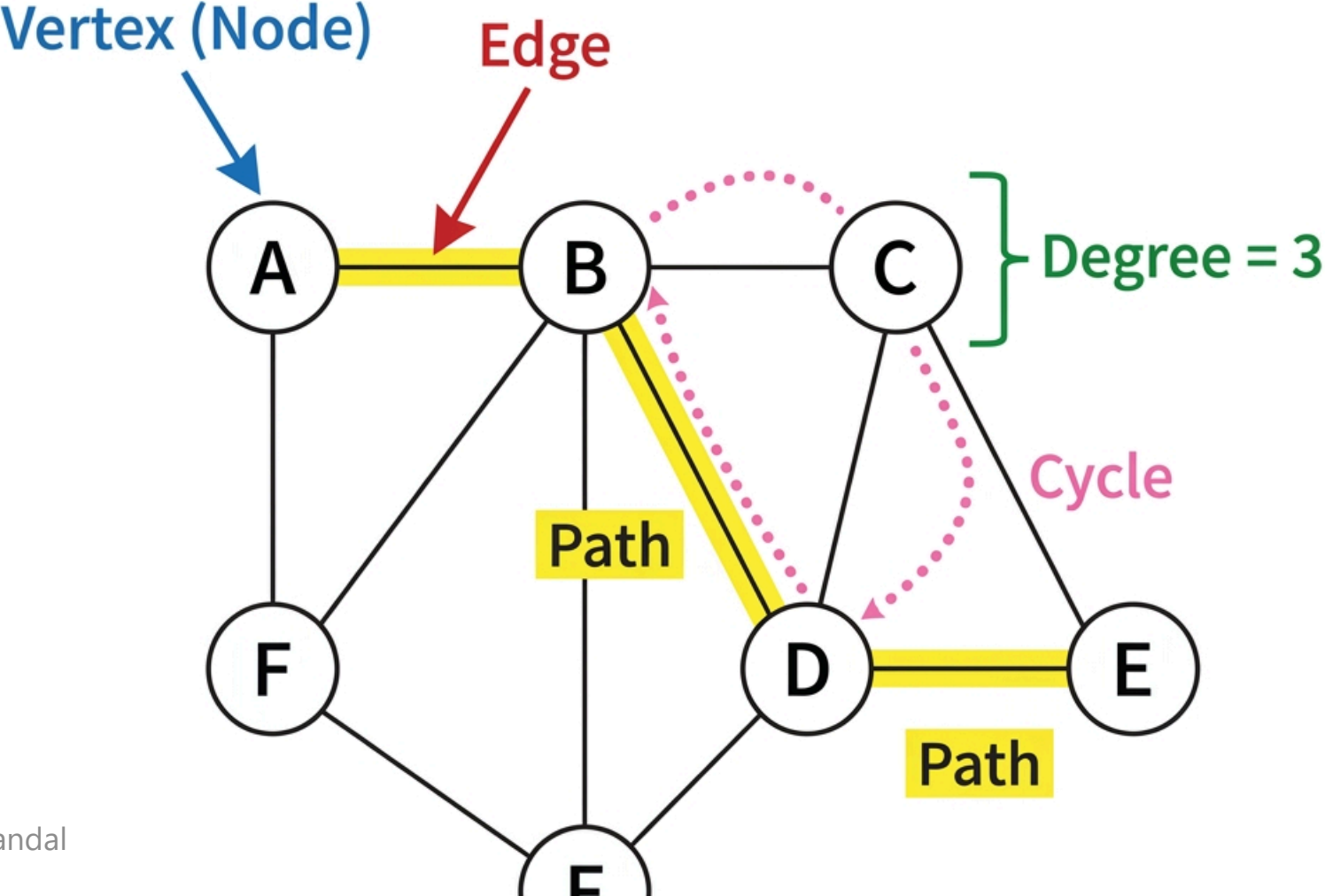
Key Insight: Every tree you have ever studied is actually just a special, restrictive case of a graph! (Trees are "connected, acyclic graphs"). This means Graphs are the superset of hierarchical structures.

Graph Terminology: The Core Vocabulary

Let's learn the language of graphs.

- **Adjacent vertices:** Two vertices are "adjacent" if they are directly connected by an edge.
- **Incident edge:** An edge is "incident" to the vertices it connects.
- **Degree of a vertex:** The number of edges connected to that vertex.
- **Path:** A sequence of vertices where each consecutive pair is connected by an edge.
- **Cycle:** A path that starts and ends at the exact same vertex.
- **Connected Graph:** A graph where there exists at least one path between *every* possible pair of vertices.
- **Disconnected Graph:** A graph partitioned into islands, where you cannot reach certain vertices from others.

Visualizing Graph Components



Part 2: Types of Graphs

Undirected vs. Directed Graphs

1. Undirected Graphs

Edges have **no direction**.

- If vertex A is connected to vertex B, it implies B is also connected to A.
- Example: *Facebook Friendships*. If I accept your friend request, we are both friends with each other simultaneously.
- Edge representation: $\{u, v\}$

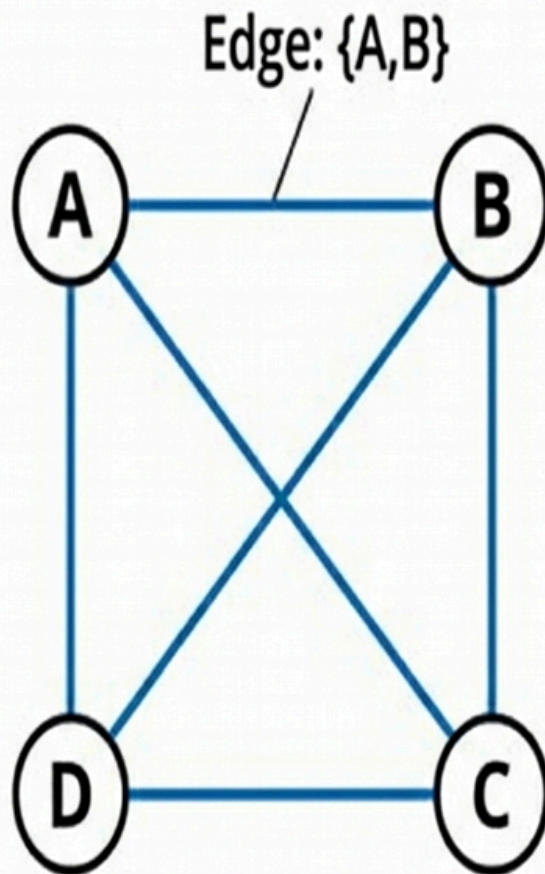
2. Directed Graphs (Digraphs)

Edges have **a specific direction**.

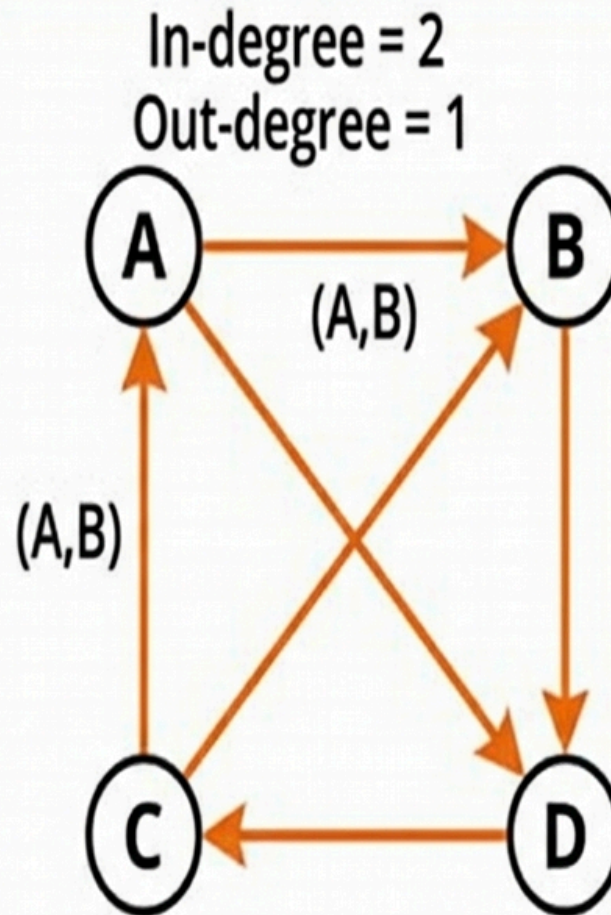
- A connection from A to B does *not* imply a connection from B to A.
- Example: *Twitter Follows*. I can follow you, but you do not have to follow me back.
- Edge representation: (u, v) (pointing from u to v).
- Has **In-degree** (arrows pointing in) and **Out-degree** (arrows pointing out).

Visualizing: Undirected vs. Directed

Undirected Graph



Directed Graph (Digraph)



Weighted vs. Unweighted Graphs

1. Unweighted Graphs

All edges are treated equally. We only care if a connection simply exists or doesn't.

- Example: "Are these two computers on the same network switch?"

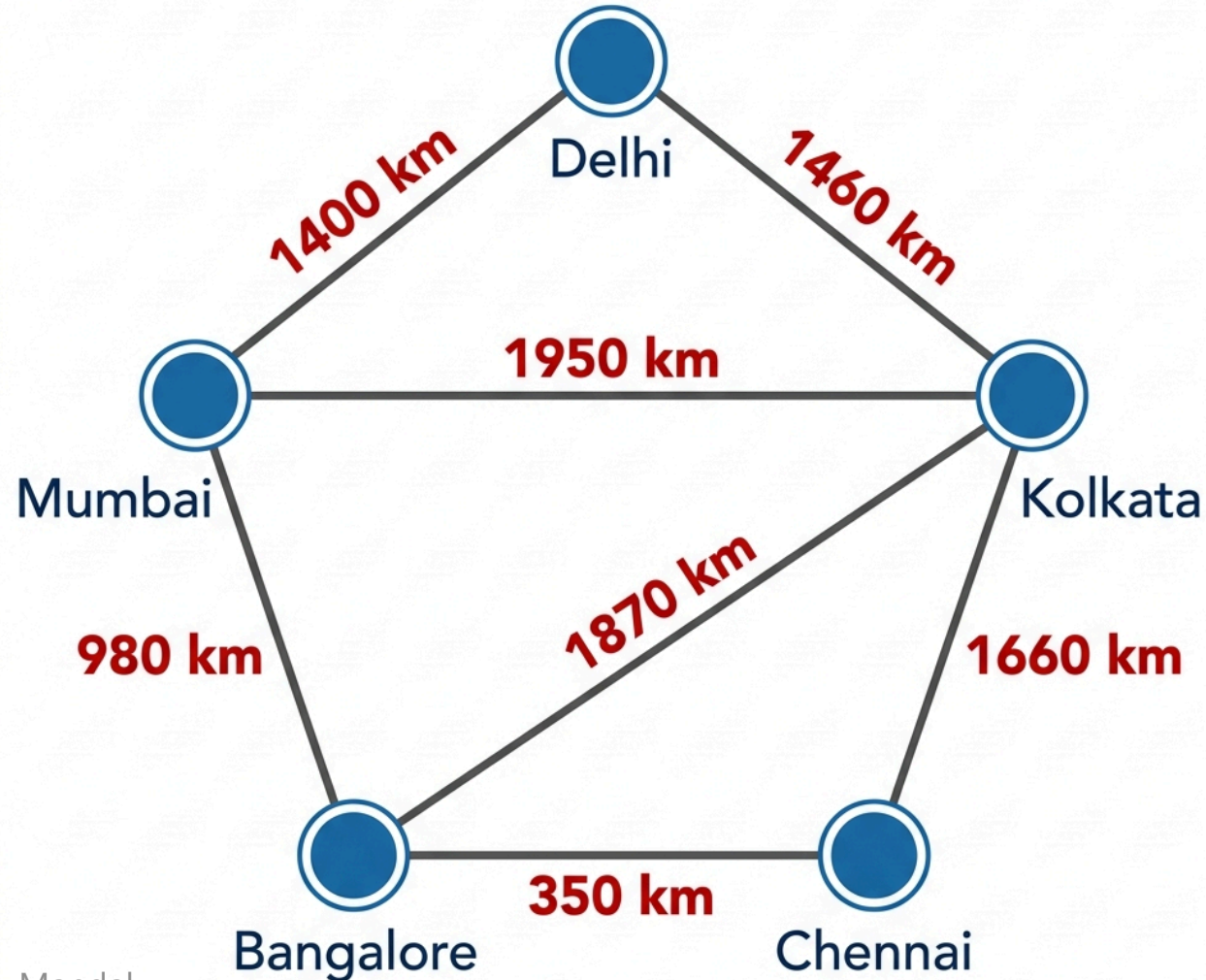
2. Weighted Graphs

Every edge is assigned a numerical **weight** (or cost, distance, capacity).

- Example: A GPS Map system. The cities are vertices, and the roads are edges. The *weight* of the edge is the physical distance (kilometers) or the time it takes to drive between them.
- Essential for *Shortest Path* operations.

Visualizing: A Weighted Graph

Weighted Graph - City Road Network



Sparse vs. Dense Graphs

How many edges does your graph have relative to the number of vertices?

- **Sparse Graph:** Has very few edges compared to the maximum possible. ($|E| \ll |V|^2$)
 - Example: A real-world map. New York is connected to a few surrounding cities, but certainly not connected directly to every other city in the world.
- **Dense Graph:** Has a large number of edges, approaching the maximum limit. ($|E| \approx |V|^2$)
- **Complete Graph (K_n):** A graph where literally *every* vertex is connected to *every* other vertex.

Note: Knowing if your graph is dense or sparse entirely changes how you should program it.

Other Important Graph Types

- **Acyclic Graph:** A graph that contains no cycles at all.
- **DAG (Directed Acyclic Graph):** A *directed* graph with no cycles -- used extensively in scheduling, build systems, and dependency management.
- **Tree:** A connected, acyclic, undirected graph with exactly $|V| - 1$ edges.

Key Relationships

- For an undirected graph: $\sum \text{degree}(v) = 2|E|$ (each edge contributes 2 to degree total)
- Maximum edges in an undirected graph: $|V|(|V| - 1)/2$
- A connected graph with exactly $|V| - 1$ edges is a tree

Key Insight: **Every tree is a graph, but not every graph is a tree.**

Part 3: Graph Representations in Memory

How Do We Code a Graph?

We can easily draw a graph on a whiteboard with circles and lines, but how do we store $G = (V, E)$ inside a computer's memory?

There are two dominant representations:

1. **The Adjacency Matrix**
2. **The Adjacency List**

Choosing carefully between such two represents the classic tradeoff between Time and Space.

Representation 1: The Adjacency Matrix

An **Adjacency Matrix** is a 2D array of size $|V| \times |V|$.

- `Matrix[i][j] = 1` (if there is an edge from vertex i to vertex j)
- `Matrix[i][j] = 0` (if there is no edge)
- *If the graph is weighted, we store the weight there instead of 1.*

For an undirected graph, the matrix will perfectly mirror itself diagonally (it is symmetric).

Adjacency Matrix: Worked Example

The Graph



Its Adjacency Matrix

	A	B	C	D	E
A	0	1	1	0	0
B	1	0	0	1	1
C	1	0	0	0	1
D	0	1	0	0	1
E	0	1	1	1	0

Reading the matrix: Edge {A,B} exists, so $\text{Matrix}[A][B] = 1$ and $\text{Matrix}[B][A] = 1$.

No edge between A and D, so $\text{Matrix}[A][D] = 0$. The matrix is **symmetric** across the diagonal.

Adjacency Matrix: Pros and Cons

Advantages:

- **Time efficiency for Lookups:** Checking if an edge exists takes $O(1)$ time.
- Dead simple to code using standard 2D arrays.

Disadvantages:

- **Incredible space waste:** It requires $O(|V|^2)$ memory to store. If we map a country with 10,000 cities, we need a matrix of 100,000,000 slots.
- But real road networks are sparse! Each city connects to maybe 4 others. Out of 100 million slots, 99.9% of them will just be $\emptyset$.
- **Finding Neighbors is Slow:** To find all neighbors for City X , we must scan the entire row X from beginning to end, taking $O(|V|)$ time.

Best for: Small graphs, or excessively Dense graphs.

Representation 2: The Adjacency List

An **Adjacency List** uses an Array of size $|V|$, where each slot holds a Linked List (or Dynamic Array).

`List[i]` contains a list of all vertices that vertex i is directly connected to.

The Math behind it

Instead of a gigantic 2D matrix, we only allocate space for the exact edges that exist.

- Index 0 $\rightarrow$ [1, 2]
- Index 1 $\rightarrow$ [0]
- Index 2 $\rightarrow$ [0, 3, 4]

Adjacency List: Pros and Cons

Advantages:

- **Space efficiency:** It takes exactly $O(|V| + |E|)$ space. If our graph of 10,000 cities only has 30,000 roads, we only allocate memory for 10,000 Array slots and 30,000 Linked List nodes. No wasted zeros!
- **Fast Traversal:** To find the neighbors of City X, we just traverse the linked list at `List[X]`. We don't have to scan empty connections. Time taken is purely proportional to the vertex's degree.

Disadvantages:

- **Slightly slower Lookup:** To find if a specific edge exists between A and B, we must traverse A's linked list checking if B is inside it. Takes $O(\text{degree})$.

Best for: Sparse graphs. (Which encapsulates 99% of real-world use cases!)

Direct Comparison

Feature	Adjacency Matrix	Adjacency List
Space Required	$O(V ^2)$	$O(V + E)$
Check if Edge exists	$O(1)$	$O(\text{degree})$
Find all Neighbors	$O(V)$	$O(\text{degree})$
Adding a new Edge	$O(1)$	$O(1)$
BFS/DFS Traversal	$O(V ^2)$	$O(V + E)$
Best used for...	Dense Graphs	Sparse Graphs

In almost all coding interviews and software engineering applications, **Adjacency Lists are the default standard** due to space constraints of the physical world.

Part 4: Graph Traversal Algorithms

Why Do We Need Traversal?

Now that we can store a graph in memory, we face the next challenge: **How do we explore it?**

Unlike trees (which have a defined root with exactly one path to each node), graphs can have:

- **Cycles** -- we could visit the same vertex over and over forever.
- **Multiple paths** between the same two vertices.

Without careful tracking, our program could run in an infinite loop!

Solution: We maintain a **visited** array or set to remember which vertices we have already seen. This prevents revisiting.

There are two fundamental philosophies for exploration: going **deep** and going **wide**.

Two Philosophies of Exploration

1. Depth-First Search (DFS): "Go deep first"

Follow one path as far as possible before backtracking. Like exploring a maze -- you walk down a corridor until you hit a dead end, then retrace your steps and try the next corridor.

2. Breadth-First Search (BFS): "Go wide first"

Explore all neighbors of the current vertex before moving deeper. Like dropping a stone in a pond -- the ripple expands outward in concentric circles.

Connection to Trees: DFS is like pre-order traversal. BFS is like level-order traversal. You have already practiced both of these in our Trees unit!

Part 4a: Depth-First Search (DFS)

DFS: The Algorithm

Core Idea: Start at a source vertex. Visit it. Then recursively visit each unvisited neighbor -- going as deep as possible before backtracking.

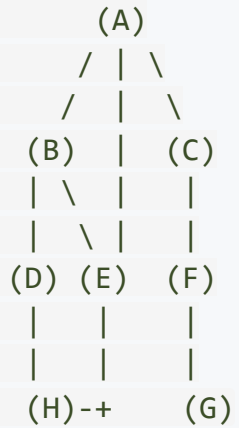
DFS uses a **Stack** -- either explicitly, or implicitly via the function call stack (recursion).

```
DFS(v):  
    mark v as visited  
    for each neighbor u of v:  
        if u is not visited:  
            DFS(u)
```

Analogy: Exploring a maze -- you follow one corridor as far as it goes, hit a dead end, backtrack to the last intersection, try the next corridor. You mark each room you visit to avoid going in circles.

DFS: Traced Example

Consider a graph with vertices A through H and these edges:



Edges: A--B, A--C, A--E, B--D, B--E, C--F, D--H, E--H, F--G

Starting DFS from vertex A (visiting neighbors in alphabetical order):

Step	Visit	Call Stack	Visited Set
1	A	[A]	{A}
2	B	[A, B]	{A, B}
3	D	[A, B, D]	{A, B, D}
4	H	[A, B, D, H]	{A, B, D, H}
5	<i>backtrack</i>	[A, B, D]	{A, B, D, H}
6	<i>backtrack</i>	[A, B]	{A, B, D, H}
7	E (skip H)	[A, B, E]	{A, B, D, H, E}
8	<i>backtrack</i> x2	[A]	{A, B, D, H, E}
9	C	[A, C]	{A, B, D, H, E, C}
10	F	[A, C, F]	{A, B, D, H, E, C, F}
11	G	[A, C, F, G]	{A, B, D, H, E, C, F, G}

DFS Order: A, B, D, H, E, C, F, G

DFS: Analysis

Time Complexity

- With **Adjacency List**: $O(|V| + |E|)$ -- we visit every vertex once and scan each edge once.
- With **Adjacency Matrix**: $O(|V|^2)$ -- for each vertex, we scan an entire row to find neighbors.

Space Complexity

- $O(|V|)$ for the visited array + recursion stack (which can grow up to $|V|$ deep in the worst case).

Applications of DFS

- **Cycle detection** -- if DFS encounters a visited neighbor that is not the parent, a cycle exists (these are called "back edges").
- **Topological sorting** of Directed Acyclic Graphs (DAGs).
- **Solving puzzles** like mazes (follow a path, backtrack on dead ends).
- **Generating random mazes** (randomized DFS).

Part 4b: Breadth-First Search (BFS)

BFS: The Algorithm

Core Idea: Start at a source vertex. Visit all its direct neighbors first. Then visit all *their* unvisited neighbors. Expand outward layer by layer.

BFS uses a **Queue** -- vertices are processed in the order they were discovered.

```
BFS(s):
    mark s as visited
    enqueue s
    while queue is not empty:
        v = dequeue()
        for each neighbor u of v:
            if u is not visited:
                mark u as visited
                enqueue u
```

Analogy: Dropping a stone in a pond -- the ripple expands in concentric circles. BFS explores the graph in "layers": first all vertices 1 edge away, then 2 edges away, and so on.

BFS: Traced Example

Using the same graph as our DFS example. **Starting BFS from vertex A:**

Step	Dequeue	Process Neighbors	Queue State	Visited
1	A	Enqueue B, C, E	[B, C, E]	{A, B, C, E}
2	B	Enqueue D (E visited)	[C, E, D]	{A, B, C, E, D}
3	C	Enqueue F	[E, D, F]	{A, B, C, E, D, F}
4	E	Enqueue H	[D, F, H]	{A, B, C, E, D, F, H}
5	D	(H visited)	[F, H]	{A, B, C, E, D, F, H}
6	F	Enqueue G	[H, G]	{A, B, C, E, D, F, H, G}
7	H	(all visited)	[G]	{A, B, C, E, D, F, H, G}
8	G	(all visited)	[]	{A, B, C, E, D, F, H, G}

BFS Order: A, B, C, E, D, F, H, G

Layer 0: {A} -- Layer 1: {B, C, E} -- Layer 2: {D, F, H} -- Layer 3: {G}

BFS: Key Property -- Shortest Path

The **BFS tree** reveals the **shortest path** (by number of edges) from the source to every other vertex.

From our example (starting at A):

- A to B: 1 edge ($A \rightarrow B$)
- A to D: 2 edges ($A \rightarrow B \rightarrow D$)
- A to G: 3 edges ($A \rightarrow C \rightarrow F \rightarrow G$)

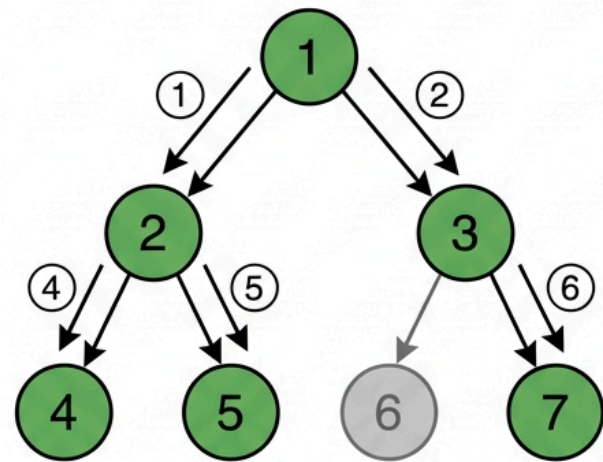
This is guaranteed because BFS visits vertices in increasing order of distance from the source.

Applications of BFS

- **Shortest path in unweighted graphs** (each edge has weight 1).
- **Social network "degrees of separation"** -- BFS from your profile gives distance to every user.
- **Web crawling** -- explore pages layer by layer from a starting URL.
- **Checking bipartiteness** (two-colorability) of a graph.

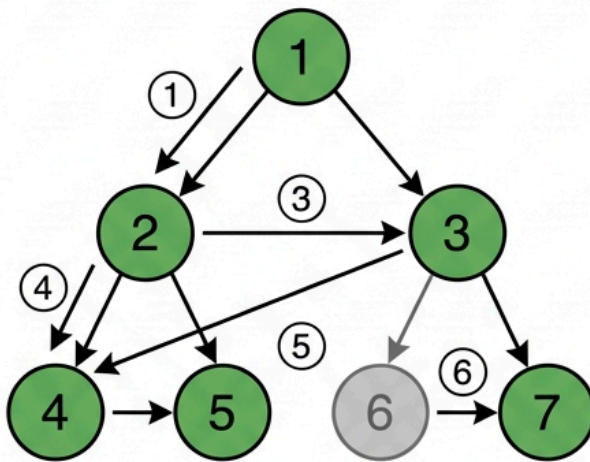
DFS vs. BFS: Visualized

DFS - Depth First Search

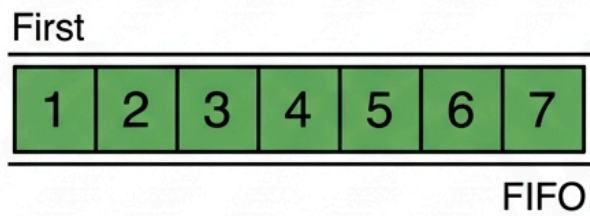
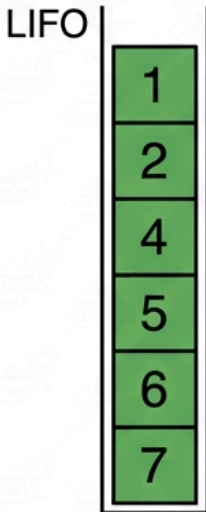


Path goes deep before backtracking.

BFS - Breadth First Search



Path explores all neighbors first.



DFS vs. BFS: Direct Comparison

Feature	DFS	BFS
Data Structure	Stack (recursion)	Queue
Exploration Style	Goes deep first	Goes wide first
Shortest Path (unweighted)	No guarantee	Yes (by edge count)
Space (worst case)	$O(V)$	$O(V)$
Time Complexity	$O(V + E)$	$O(V + E)$
Cycle Detection	Yes (via back edges)	Yes (via cross edges)
Best For	Cycle detection, topological sort, mazes	Shortest path, connectivity, level-order

Both have **identical time complexity**. Your choice depends on the *problem* you are solving.

Part 5: Connected Components

What is a Connected Component?

A **connected component** is a maximal set of vertices such that there is a path between every pair within the set.

- A graph can have **one or more** connected components.
- If the entire graph is connected, it has exactly **one** connected component.
- If it has disconnected "islands," each island is a separate component.

Example: A social network where two groups of people have zero connections between them -- these are two separate connected components.

Finding Connected Components

We can find all connected components using repeated DFS (or BFS):

```
component_count = 0
for each vertex v in the graph:
    if v is not visited:
        component_count += 1
        DFS(v)  // or BFS(v) -- marks all reachable vertices as visited
```

Worked Example (10 vertices, 3 components)

- DFS from vertex 0 reaches {0, 1, 2, 3} -- **Component 1**
- Vertex 4 is unvisited. DFS from vertex 4 reaches {4, 5} -- **Component 2**
- Vertex 6 is unvisited. DFS from vertex 6 reaches {6, 7, 8, 9} -- **Component 3**

Time Complexity: $O(|V| + |E|)$ -- every vertex and edge is visited exactly once across all DFS/BFS calls.

Application: Network reliability -- "If a server goes down, does the network split into disconnected parts?"

Unit Summary & Wrap Up

Wrap Up

Let's summarize everything we covered today:

Graphs -- Structure:

1. Built via **Vertices** (entities) and pairwise **Edges** (connections).
2. Can be **Directed** (one-way streets) vs **Undirected** (mutual).
3. Can be **Weighted** (cost/distance) vs **Unweighted**.

Graphs -- Storage:

4. **Adjacency Lists** for large sparse graphs, **Adjacency Matrices** for small dense mappings.

Graphs -- Exploration:

5. **DFS** (Stack) -- go deep, backtrack. Powers cycle detection and topological sort.
6. **BFS** (Queue) -- go wide, layer by layer. Finds shortest paths in unweighted graphs.
7. **Connected Components** -- run DFS/BFS from each unvisited vertex.

Where to Next?

Today we loaded graphs into memory and learned to explore them with DFS and BFS.

But what happens when edges have **weights** and we need to find the *cheapest* way to connect everything, or the *shortest* route between two cities?

Next lecture, we tackle the grand finale of graph algorithms:

- **Minimum Spanning Trees** (Prim's and Kruskal's Algorithms)
- **Shortest Path** (Dijkstra's Algorithm)

These algorithms power GPS navigation, network routing, and infrastructure planning -- the engines of optimization on graphs.